Módulo 2 - Grupo: JOULIE FARADAY

2026-04-15

Introducción

Este es un documento en R Markdown. Permite generar HTML o Word combinando texto y código. El lenguaje R es una poderosa herramienta matemática y estadística. En este reporte generado con R Markdown, analizaremos la resolución de distintos problemas y evaluaremos la calidad de diferentes algoritmos implementados.

Probando primeros comandos de R

R es un leguaje muy parecido a matlab y trabaja con variables que en general pueden ser matrices.

```
A <- 38
A <- 40
B <- 60
C <- B - A
C
```

```
## [1] 20
```

Uso de data sets o base de datos internos de R

Usando el comando data() en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos estan cargados en tablas de las cuales podemos elegir columnas especificas si escribimos el signo $ despues del comando de informacion especifico que queremos.

También está el comando summary(), que es una de las herramientas más útiles y rápidas que tiene R para hacer estadística descriptiva básica.

```
# Planteamos el dataset
data(cars)
```

```
# Usamos el símbolo $ para extraer SOLO la columna de velocidades (speed)
velocidades <- cars$speed

# Mostramos los primeros 6 valores de esa columna aislada
head(velocidades)
```

```
## [1] 4 4 7 7 8 9
```

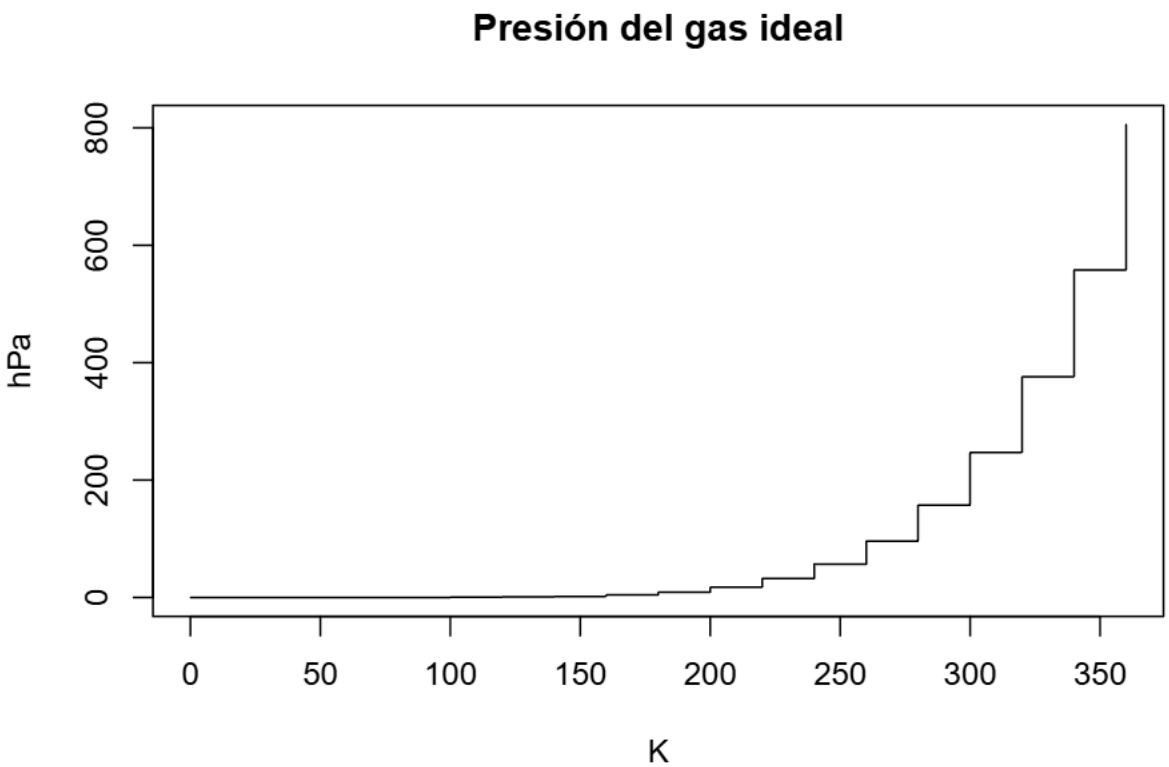
```
summary(cars)
```

```
##      speed           dist
##  Min.   : 4.0   Min.   :  2.00
##  1st Qu.:12.0   1st Qu.: 26.00
##  Median :15.0   Median : 36.00
##  Mean   :15.4   Mean   : 42.98
##  3rd Qu.:19.0   3rd Qu.: 56.00
##  Max.   :25.0   Max.   :120.00
```

Comando plot (Gráficos)

Es un comando que sirve para graficar cualquier fuente de datos que le asignemos: Tenemos la posibilidad de asignar nombres a las variables o hacer cambios del grafico, en caso de no saber el comando podemos escribir *plot* en la consola, a la derecha nos va a abrir una pestaña y ahí tocar Generic X-Y Plotting y ahi nos va a explicar cada comando

```
plot(pressure, type = "s",
     main = "Presión del gas ideal",
     ylab = "hPa",
     xlab = "K")
```



Funciones estadísticas

Otro comando usado es el comando rnorm. Este lo que hace es generar datos aleatorios, pero forzando a que esos datos sigan la forma de una Distribución Normal (la famosa campana de Gauss).

Podemos asignarle ciertos parámetros si queremos: Tamaño de la muestra * **n* (es la cantidad de datos que le pides a R que invente, en este caso 350) Media (el valor central, en este caso 22), o Desviación estándar (Representa la dispersión o el "error experimental", en este caso, 5).

```
z1 <- rnorm(350, 22, 5)
z1
```

```
##   [1] 19.916986 16.093351 25.165518 19.240628 22.664938 15.001690 28.638654
##   [8] 28.156368 22.894132 28.400053 28.047981 21.807166 12.506754 25.795555
##  [15] 28.564598 27.374650 23.575349 26.342430 12.019170 14.092052 12.126106
##  [22] 21.528077 23.866738 20.246818 19.604015 25.107917 20.744456 17.003657
##  [29] 22.501724 14.188301 19.116235 24.471662 21.021480 25.983178 15.775986
##  [36] 11.412034 28.848065 20.428825 24.992568 19.250844 26.990446 25.061312
##  [43] 28.884675 17.569700 26.232528 27.086394 22.739250 18.042981 21.964040
##  [50] 18.487563 19.729578 22.390640 23.002060 23.296506 25.805632 18.528206
```

```
##  [57] 19.520530 28.137641 24.405664 20.932387 24.149414 29.459854 23.486517
##  [64] 34.065520 23.202610 18.176764 17.647545 16.920869 25.293236 23.194323
##  [71] 21.326974 22.063188 21.459351 22.659985 26.805636 27.016242 16.242087
##  [78] 23.553575 25.445609 21.069127 21.269643 28.158370 26.975003 19.354001
##  [85] 15.733832 21.974702 27.276705 25.355915 25.196207 22.437504 34.107368
##  [92] 29.412577 22.881847 26.550705 22.666768 18.830659 25.193764 17.127008
##  [99] 20.093550 33.353960 16.337499 22.565678 15.798687 19.133158 32.774686
## [106] 18.031847 24.819063 21.320111 14.538938 29.660510 23.826207 17.005330
## [113] 28.060535 18.518378 20.059755 11.564916 12.275077 21.815573 15.503699
## [120] 19.754059 28.882073 24.577647 22.826301 12.547640 26.189381 19.690374
## [127] 13.270936 11.960277 26.532320 18.090850 21.428127 26.444480 24.953024
## [134] 18.035701 17.743580 29.666866 15.870911 29.429356 17.393902 20.695491
## [141] 25.710632 19.645337 16.453389 20.114706 20.292323 16.136196 28.825600
## [148] 17.731554 16.199262 24.492651 25.216626 20.729453 19.772759 25.573220
## [155] 28.457620 19.190030 15.225228 27.261857 21.805505 27.607408 24.620703
## [162] 14.854981 25.194536 20.447783 27.372336 19.773135 29.340223 20.480424
## [169] 24.840593 20.109164 25.129801 15.577240 24.700538 17.280763 27.673465
## [176] 23.435309 13.042216 29.282559 23.822317 22.625775 19.273060 27.633480
## [183] 23.667666 27.368681 16.426127 22.102406 22.082074 23.371032 28.563011
## [190] 15.796962 19.517966 19.602432 23.428889 23.018593 28.357138 20.544474
## [197] 27.108641 21.942077 18.919459 15.439863 30.518069 23.782116 29.675747
## [204] 21.903571 19.083806 24.763558 28.965714 19.983634 12.545476 27.291263
## [211] 29.795723 19.283329 22.141652 16.034869 32.738584 16.312463 27.449417
## [218] 28.723289 17.225100 18.705307 33.624401 23.472648 20.210499 18.874663
## [225] 23.207611 27.662579 21.652623 22.213024  7.931856 30.385391 23.247253
## [232] 21.673475 23.644906 22.120228 22.240970 29.975202 28.039636 20.208494
## [239] 29.159179 20.787207 28.259774 18.910293 26.054295 23.531668 19.357485
## [246] 15.486672 22.834725 25.710321 29.271932 16.064127 18.861626 12.615109
## [253] 14.640345 32.053384 12.398073 26.531115 26.351235 17.250331 19.820367
## [260] 11.377785 18.398686 28.357497 17.875570 13.500728 21.177196 24.490363
## [267] 19.022743 21.244180 26.880681 23.052943 18.684739 21.236019 20.529354
## [274] 24.630685 10.792075 19.463309 13.629377 19.915738 22.295915 24.975754
## [281] 20.561957 24.207043 25.333096 24.865974 31.431417 18.369960 27.347808
## [288] 17.466777 21.415388 18.121965 19.893537 17.948344 28.409017 12.293497
## [295] 20.180809 14.908353 18.385715 21.490870 19.358719 28.470010 24.721141
## [302] 16.461958 15.841331 21.423771 18.330058 23.524387 27.095207 14.517683
## [309] 21.855527 17.136540 21.836841 24.684643 21.448486 25.313194 10.306502
## [316] 18.546068 25.335800 25.269879 19.081045 28.439873 21.785759 24.985287
## [323] 18.187778 17.050195 13.474888 16.236395 28.059499  8.498531 27.158173
## [330] 21.929949 22.903936 21.332475 27.404581 19.281548 14.506505 18.189609
## [337] 25.241798 18.580876 13.071482 28.823613 23.034195 23.646594 20.547649
## [344] 24.539510 24.488650 31.707880 25.357515 23.142500 25.402604 20.834961
```

```r
w1 <- length(z1)
w1
```

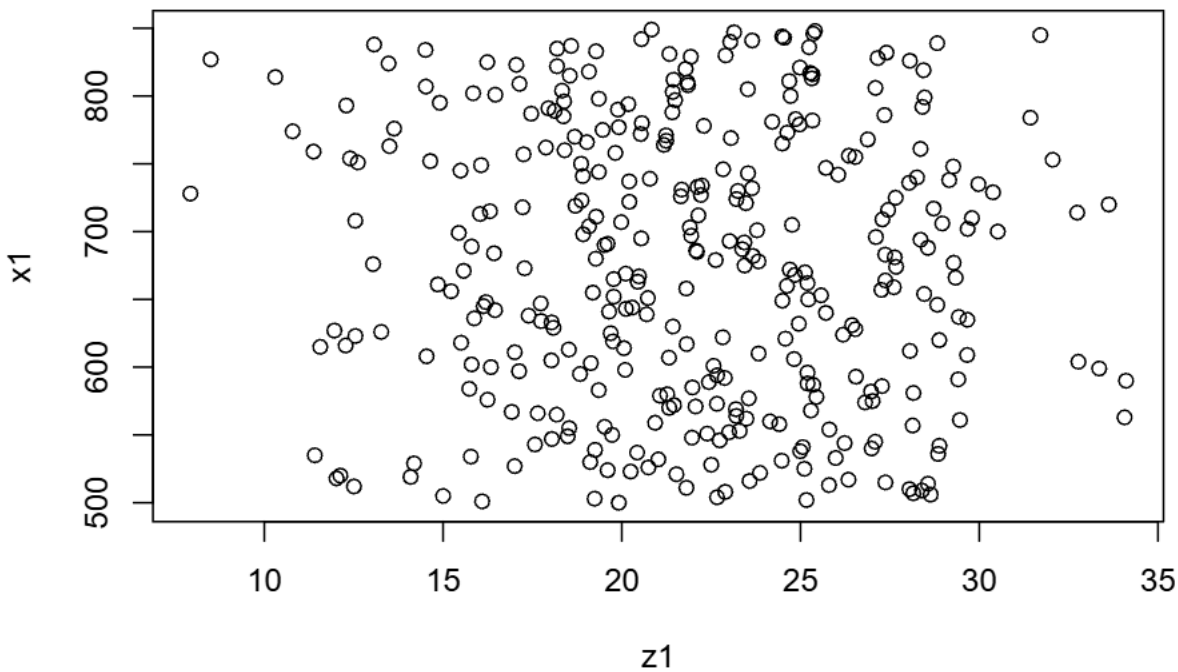
```
## [1] 350
```

```r
x1 <- 500:849
x1
```

```
##   [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
##  [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
##  [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
##  [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
##  [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
```

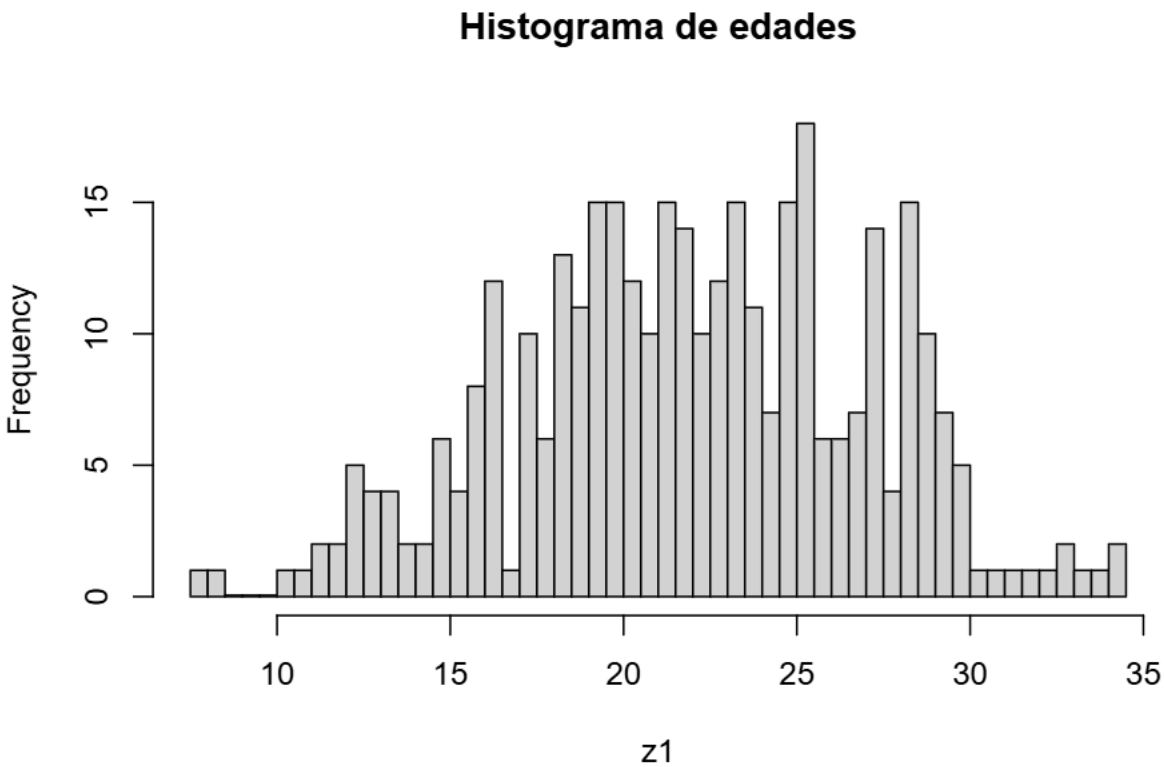
```
##  [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849
```

```r
plot(z1, x1)
```



El comando hist() genera un histograma.

```r
hist(z1, main = "Histograma de edades", breaks = 60)
```



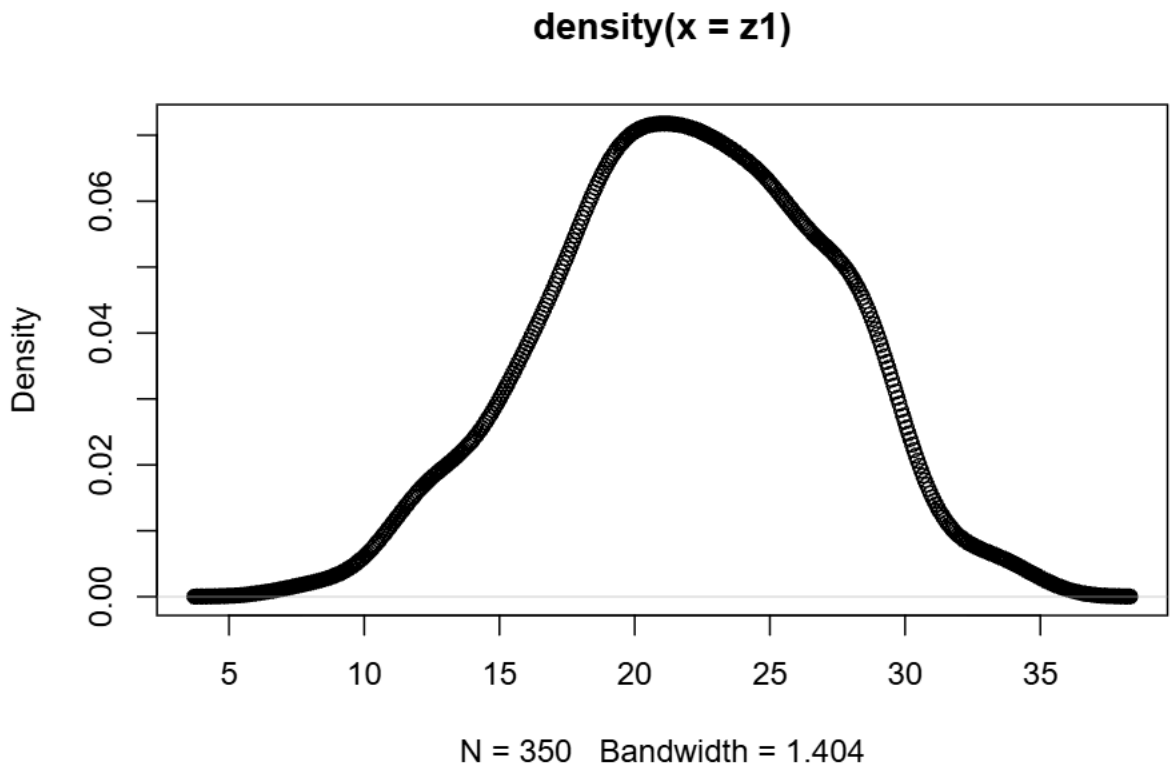
El comando density() calcula la densidad de probabilidad. Es la versión matemática, suavizada y continua del histograma.

```
density(z1,type="b")
```

```
## Warning: In density.default(z1, type = "b") :
##  extra argument 'type' will be disregarded
```

```
##
## Call:
##  density.default(x = z1, type = "b")
##
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.404
##
##        x                y
##  Min.   : 3.721   Min.   :0.0000116
##  1st Qu.:12.370   1st Qu.:0.0031514
##  Median :21.020   Median :0.0203933
##  Mean   :21.020   Mean   :0.0288473
##  3rd Qu.:29.669   3rd Qu.:0.0553368
##  Max.   :38.318   Max.   :0.0717867
```

```
plot(density(z1),type="b")
```



Ejercicio 1: Designación

Consigna: Las ultimas 3 cifras del DNI son 803, crear una variable que tenga ese número

```
DNI <- 803
```

Consigna: Crear un vector del 1 al 803.

```
secuencia_dni <- 1:803
secuencia_dni
```

```
##   [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
##  [19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
##  [37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
##  [55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
##  [73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
##  [91]  91  92  93  94  95  96  97  98  99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
## [181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198
## [199] 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216
## [217] 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234
## [235] 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252
## [253] 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270
## [271] 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288
## [289] 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306
## [307] 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324
## [325] 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342
```

```
## [343] 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360
## [361] 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378
## [379] 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396
## [397] 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414
## [415] 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432
## [433] 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450
## [451] 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468
## [469] 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486
## [487] 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504
## [505] 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522
## [523] 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540
## [541] 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558
## [559] 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576
## [577] 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594
## [595] 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612
## [613] 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630
## [631] 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648
## [649] 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666
## [667] 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684
## [685] 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702
## [703] 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720
## [721] 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738
## [739] 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756
## [757] 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774
## [775] 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792
## [793] 793 794 795 796 797 798 799 800 801 802 803
```

Ejercicio 2: Uso de for

Consigna: Calcular la suma de todos los valores del vector secuencia_dni usando un for:

```r
Total <- 0
valor_final <- length(secuencia_dni)

for (i in 1:valor_final) {
  Total <- Total + i
}
Total
```

```
## [1] 322806
```

Ejercicio 3: Phyton

Consigna: Repetir el ejercicio anterior pero en Phyton

```python
# --- Ejercicio 1 ---
# Consigna: Crear una variable con las últimas 3 cifras del DNI (803)
dni = 803

# Consigna: Crear un vector (lista) del 1 al 803
# En Python, range(1, 804) genera números del 1 al 803
secuencia_dni = list(range(1, dni + 1))
# --- Ejercicio 2 ---
```

```python
# Calcular la suma de todos los valores usando un bucle 'for'
total = 0

for i in secuencia_dni:
    total += i

# Mostrar el resultado final
print(f"La suma total es: {total}")
```

```
## La suma total es: 322806
```

Ejercicio 4: Medición de tiempo (sys.time)

Consigna:¿Cuanto tarda en correr el código del ejercicio 2?

Usamos el comando *sys.time*

```r
inicio <- Sys.time()
total <- 0
valor_final <- 10000000*length(secuencia_dni)
for (i in 1:valor_final)
total <- total + i
total
```

```
## [1] 3.224045e+19
```

```r
final <- Sys.time()
final-inicio
```

```
## Time difference of 2.288593 mins
```

Ejercicio 5: 2da medición de tiempo (tictoc)

Consigna: Aplicar la consigna anterior pero aprendiendo a usar el comando *tictoc*

```r
library(tictoc)

# Iniciamos el cronómetro asignándole un nombre descriptivo a la prueba
tic("Tiempo de ejecución")

# Inicio del bloque a medir
Total<-0
valor_final<-length(secuencia_dni)

for(i in 1:valor_final) {
Total<-Total + i
}

# Fin del bloque a medir

#Se "detiene" el cronómetro.

#Esto imprimirá el tiempo transcurrido en pantalla.
toc()
```

```
## Tiempo de ejecución: 0.004 sec elapsed
```

```r
#Imprimimos la variable Total para verificar que el cálculo matemático se hizo bien
Total
```

```
## [1] 322806
```

Ejercicio 6: Secuencia

Consigna: Generar secuencia de 2 en 2 hasta 50000

```r
inicio_for <- Sys.time()

A <- numeric(50000)
for (i in 1:50000) {
  A[i] <- i * 2
}

final_for <- Sys.time()
tiempo_for <- final_for - inicio_for
```

Secuencia con función de R:

```r
inicio_seq <- Sys.time()

B <- seq(2, 100000, by = 2)

final_seq <- Sys.time()
tiempo_seq <- final_seq - inicio_seq


# Resultados
tiempo_for
```

```
## Time difference of 0.006720781 secs
```

```r
tiempo_seq
```

```
## Time difference of 0.001932144 secs
```

Ejercicio 7: Serie de Fibonacci

Consigna: Generar sucesión o serie de Fibonacci hasta superar 1.000.000

```r
# Generar Fibonacci hasta superar 1.000.000

fibonacci <- c(0,1)
iteraciones <- 2

while (fibonacci[length(fibonacci)] <= 1000000) {
  nuevo <- fibonacci[length(fibonacci)] + fibonacci[length(fibonacci)-1]
  fibonacci <- c(fibonacci, nuevo)
  iteraciones <- iteraciones + 1
}
```

```r
fibonacci
```

```
##  [1]       0       1       1       2       3       5       8      13      21
## [10]      34      55      89     144     233     377     610     987    1597
## [19]    2584    4181    6765   10946   17711   28657   46368   75025  121393
## [28]  196418  317811  514229  832040 1346269
```

```r
iteraciones
```

```
## [1] 32
```

Ejercicio 8: Definición matemática recurrente

Consigna:¿Cuantas iteraciones se necesitan para generar un número de la serie mayor que 1.000.000?

```r
f0<-0   #Valores iniciales
f1<-1
f2<-0
i<-0  #Número de iteraciones incial
valorfinal<-1000000 #Umbral

while (f2<valorfinal) {

  f2=f0+f1
  f0=f1
  f1=f2
  i<-i+1

}
f2
```

```
## [1] 1346269
```

```r
i
```

```
## [1] 30
```

Ejercicio 9: Método Burbuja (Bubble Sort)

Consigna: Compara la performance de ordenación del método burbuja vs el método sort de R.

```r
x <- sample(1:20000, 20000)

burbuja <- function(x) {
  n <- length(x)
  for (j in 1:(n-1)) {
    for (i in 1:(n-j)) {
      if (x[i] > x[i+1]) {
        temp <- x[i]
        x[i] <- x[i+1]
        x[i+1] <- temp
      }
    }
  }
```

```
  return(x)
}
```

Comparación:

```
system.time({
  res1 <- burbuja(x)
})
```

```
##    user  system elapsed
##  20.764   0.024  20.899
```

```
system.time({
  res2 <- sort(x)
})
```

```
##    user  system elapsed
##   0.001   0.000   0.000
```

Ejercicio 10: Penitencia de Newton

Consigna: Desarrollar dos algoritmos que hagan el trabajo de sumar desde 1 hasta 1000000 y verifcar cuál de los dos es más eficiente

Método 1 (for):

```
n <- 1000000

system.time({
  suma1 <- 0
  for (i in 1:n) {
    suma1 <- suma1 + i
  }
})
```

```
##    user  system elapsed
##   0.017   0.000   0.017
```

Método 2 (fórmula matemática):

```
system.time({
  suma2 <- n * (n + 1) / 2
})
```

```
##    user  system elapsed
##       0       0       0
```

Comparación:

```
suma1
```

```
## [1] 500000500000
```

```
suma2
```

```
## [1] 500000500000
```

Conclusión final:

A lo largo de este trabajo, evaluamos la eficiencia de distintos algoritmos en R mediante técnicas de benchmarking (como Sys.time, tictoc y microbenchmark). Comprobamos de manera empírica que un código no es de calidad solo por llegar al resultado correcto, sino por cómo optimiza el uso del procesador y el tiempo de ejecución.Por ejempolo, el ejercicio de "La penitencia de Newton" demostró este principio claramente.

Mientras un bucle iterativo requiere un costo computacional inmenso para realizar operaciones masivas, aplicar un buen método matemático resuelve el mismo problema de forma casi instantánea.

La medición de tiempos que utilizamos lo confirma: la herramienta de optimización más poderosa en la programación no es el hardware, sino el razonamiento físico y matemático previo. Los modelos analíticos demuestran ser siempre abrumadoramente más eficientes que la iteración tradicional.